

GEO1004 Assignment 3: Simple Polyhedron Processing

15 June 2026

Ruben Vons
6285880

Artemi Kurski
6195784

Moritz Cermann
6562906

1. Introduction

This report describes our approach for the 3rd assignment of the GEO1004 course at TUDelft. In it, we convert BIM — specifically IFC — objects to CityJSON.

In the conversion, we perform an intermediate step — voxelisation, from which we later recover the building geometries and rooms. Voxelisation allows us to recover valid geometry even when the geometry of the input file is invalid — e.g. non-manifold or non-watertight — and correctly identify rooms even when the corresponding IfcSpace does not match the room geometry¹.

We test our code for 2 files — the IfcOpenHouse² and the Wellness Centre Sama³.

2. Methodology

Overall, the steps we take follow the structure suggested in the assignment:

1. Convert IFC to OBJ;
2. Read OBJ and convert it to CGAL geometry;
3. Voxelise the geometry;
4. Extract isosurfaces from the geometry;
5. Detect individual rooms;
6. Write the output to a CityJSON file.

2.1. IFC to OBJ Conversion

For simplicity and since we are not explicitly interested in all IFC semantics, we convert the .ifc file to a .obj file using IfcConvert utility⁴. The general principle we follow is to keep the objects which form the building envelope — walls, floors, roofs, windows, doors etc — and discard those that do not — such as furniture or outdoor objects.

2.2. OBJ Reader

The structure of the converted IFC file differs slightly from previous implementations of OBJ files we have worked with thus far. The general structure still consists of faces and vertices. The converted file also

includes vertex normals (vn), shells (usemtl), object groups (g) and faces that list both vertices and vertex normals. As a basis, we took the OBJ reader code from assignment 1 and adapted it in a way that discards vertex normals as those will be recalculated after the voxelisation by 3D Isosurfacing. We save the contents of this OBJ file as a list of vertices and another list which contains for each object its shells, and for each shell its faces in CGAL's Triangle_3 format.

2.3. Voxelisation

For the voxel structure we use the example code that was given in the assignment. In the code, it is represented with a structure VoxelGrid, which allows to write and retrieve values assigned to a voxel at given coordinates as VoxelGrid(x, y, z).

To construct it we require the extents of the voxel grid, as well as the dimensions (dim) of a voxel. For this we can use the 3D bounding box of our set of vertices. This gives us the minimum and maximum dimensions in x, y and z directions, and we initialise the voxel grid with dimensions $\frac{x}{\text{dim}}, \frac{y}{\text{dim}}, \frac{z}{\text{dim}}$. For the definition of the rooms we want to first be able to separate outside from inside for the structure. To ensure the connectivity of the outside air in case of a flood fill, a buffer of 2 voxels is added in each dimension. By setting the Cartesian origin (the minimum corner of the bbox), we ensure that our object is entirely surrounded by empty voxels.

In addition, we can convert the voxel centres back to Cartesian coordinates using the voxel_to_world function, which takes the voxel position, minimum x, y and z position and the size of a voxel.

While perhaps redundant, we use the object list to extract all triangles and store them into a new list. We start off by looping through each triangle. By taking the minimum and maximum extent of the triangle we can determine the bounding box in voxel space. This is useful as it means we only have to loop through the relevant voxel ranges instead of the entire space.

To ensure 26-connectivity, we set the intersection target to a solid. As CGAL's do_intersect function does not support solids, we approximate it with an instance of Bbox_3 — the logic being that if any point

¹examples of both can be found in the *Applications of 3D modelling of the built environment* lecture delivered as part of GEO1004 course in 2026: <https://3d.bk.tudelft.nl/courses/geo1004/data/apps.pdf>

²found here: <https://github.com/aothms/IfcOpenHouse>

³found here: <https://openifcmodel.cs.auckland.ac.nz/>

⁴see <https://docs.ifcopenshell.org/ifcconvert/usage.html>

of a triangle intersects the boundary or the interior of a bounding box, the object is represented by a voxel.

If the triangle intersects the voxel will be considered to be a building boundary (e.g. a wall, a roof, or a floor), and we write 1 to its coordinates in the VoxelGrid. We note that switching to a b-rep data structure with OBJ means that a small enough voxel will result in the void between two surfaces of e.g. a wall being treated as a room.

2.4. Room Detection

We have attempted several methods for room detection — the details can be found in Section 4.4. The approach we settled on is flood-filling both the exterior and the rooms:

FLOOD FILL

```

Input: grid: 3D-array of IDs corresponding to
voxelised objects, where 0 is unvisited, 1 is part
of building, and other positive integers corre-
spond to different rooms
Output: 3D-array of IDs corresponding to vox-
elised objects
1 region ID:  $i \leftarrow 2$ 
2  $x_{\max}, y_{\max}, z_{\max} \leftarrow$  dimensions of voxel grid
3 for  $x \in [0, x_{\max}] \mid \mathbb{N}$  do:
4   for  $y \in [0, y_{\max}] \mid \mathbb{N}$  do:
5     for  $z \in [0, z_{\max}] \mid \mathbb{N}$  do:
6       if grid[ $x, y, z$ ]  $\neq 0$  do:
7         | continue
8         add ( $x, y, z$ ) to queue
9         while queue has items do:
10           $x_{\text{curr}}, y_{\text{curr}}, z_{\text{curr}} \leftarrow$  front of queue
11          pop front of queue
12          query 6 neighbours of  $x_{\text{curr}}, y_{\text{curr}}, z_{\text{curr}}$ 
13          if grid[neighbour] = 0 do:
14            | add neighbour to queue
15            grid[ $x_{\text{curr}}, y_{\text{curr}}, z_{\text{curr}}$ ]  $\leftarrow i$ 
16           $i \leftarrow i + 1$ 

```

The code for the exterior and the room fill is slightly different — the exterior seed is fixed at 0, 0, 0 — but the general principle holds.

2.5. Surface Cleanup and Extraction

We remove surfaces which are only one voxel tall — these do not represent usable space and instead are an artifact due to thick floor slabs. We filter them out based on bounding box dimensions.

After trying several approaches to surface extraction (see Section 4.3 for details), we have settled on simply extracting the voxelisation. The logic is as follows:

EXTRACTING VOXEL SURFACES

```

Input: grid: 3D-array of IDs corresponding to vox-
elised objects
Output: Meshes of voxelised surfaces
1 region ID:  $i \leftarrow 2$ 
2  $x_{\max}, y_{\max}, z_{\max} \leftarrow$  dimensions of voxel grid
3 for each voxel do:
4   if grid( $x, y, z$ ) is not part of building (solid) do:
5     | continue
6     for each of 6-neighbours do:
7       if a neighbour is a solid do:
8         | continue — skips boundaries between
9         | building voxels
10      if a neighbour is exterior do:
11        | add corresponding surface to exterior
12        | boundary
13      if a neighbour is interior do:
14        | add corresponding surface to the interior
15        | boundary with corresponding ID

```

Additionally, we assign rooms to storeys. For this, we extract the floor level (i.e. height of the lowest voxel) for each room; sort them by height; and assign a room to a taller storey whenever there is a gap in the heights. The exact size of the gap — and if the filtering is necessary at all — depends on the dataset. We find it to be beneficial for the Wellness Centre Sama (with a gap of 2) but not for the OpenHouse.

2.6. Saving to CityJSON

There are several key steps we take when converting the meshes to CityJSON for both geometry and semantics.

2.6.1. Geometry Conversion

CityJSON uses a point dictionary model for storing polygons. To convert to point dictionary, we:

1. Write all vertices of a boundary to a single list;
2. Store an offset equal to the number of vertices for each boundary;
3. Convert the list of vertices from float by dividing the coordinates by a scaling factor;
4. For each boundary, store the list of vertex indices + offset.

2.6.2. Storing Objects

We store:

- Building

- geometry
 - type is multisurface
 - LoD is 1
 - boundaries reference vertices
- Building storey
 - Attribute: storey number
 - Parent: building
- Building room
 - geometry
 - type is multisurface
 - LoD is 1
 - boundaries reference vertices
 - Parent: building storey

3. Evaluation

In this section, we evaluate the performance of our conversion and room detection pipeline using two distinct building models. We test our approach on both a simple, single-room structure and a more complex, multi-storey facility to assess the robustness of our voxelisation, isosurface extraction, and room detection methodologies.

3.1. Input file

For testing we used two IFC models: `IfcOpenHouse_IFC2x3.ifc`, a simple single-room building with a thick foundation, and `2022020320211122Wellness center Sama.ifc`, a three storey building with multiple rooms, a staircase and a flat roof.

We noticed, that the Wellness Center model has a few small openings where the glass facade meets the ceiling slabs, plus some overlapping geometry. However, we don't think this affected our results, since these gaps are small compared to our voxel size and did not cause any unwanted leaks in the flood fill.

Now, to convert each IFC model to OBJ format we used `IFCConvert` to keep only the entity types that are relevant to the building form and dropping furniture, street level context and other non-structural elements. For the Wellness Center these relevant entities included: `IfcWall`, `IfcRoof`, `IfcFooting`, `IfcWindow`, `IfcDoor`, `IfcStairFlight`, `IfcSlab`, `IfcMember`, and `IfcPlate`. We also excluded four elements by their name. The full conversion command can be found in the `README.md` file.

3.2. Results

In this section we present results for the experiments that were conducted for the different extraction and conversion methods on both the `IfcOpenHouse` and the `Wellness Center`.

3.2.1. House

Figure 1 shows the OBJ model of the house after conversion. The house is a connected solid mass, with no separate pieces such as door frames, columns or open windows. Figure 2 shows the results of the mesh-based room extraction approach (see Section 4.4.3), conducted with a voxel size of 0.5m. We find two interior rooms, and the mesh splits into 3 parts: one outer shell and two inner parts.

Figure 3 displays the results obtained via the voxel-based surface extraction (see Section 4.4.2). The left image uses a voxel size of 0.5m, while the right shows the same method applied at a resolution of 0.25m. Here extra “rooms” appear. These are not real rooms. They are gaps inside the exterior wall itself, between its inner and outer layer. At 0.25m this gap is wide enough for flood fill to treat it as its own enclosed space.

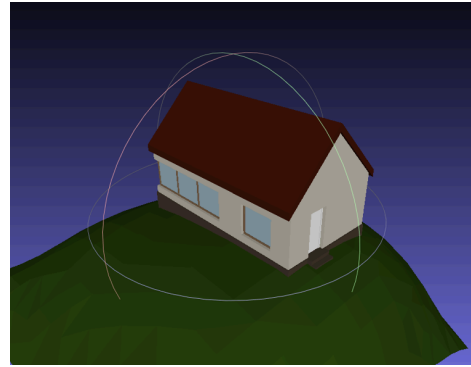


Figure 1: The `IfcOpenHouse` model after conversion to OBJ. The house is a single connected solid mass, with no separate door frames, columns, or open windows.

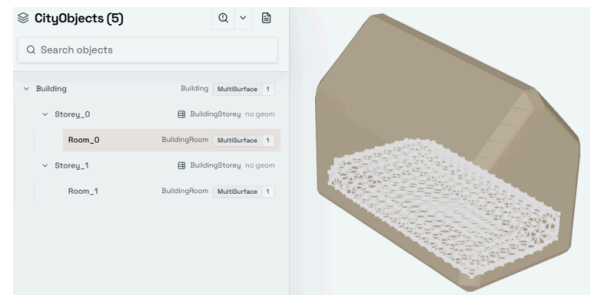


Figure 2: Result of the mesh-based room extraction (Section 4.4.3) for the House at voxel size 0.5m. The output contains 5 CityObjects: the Building, two BuildingStorey objects, and two BuildingRoom objects, matching the house's two interior rooms.

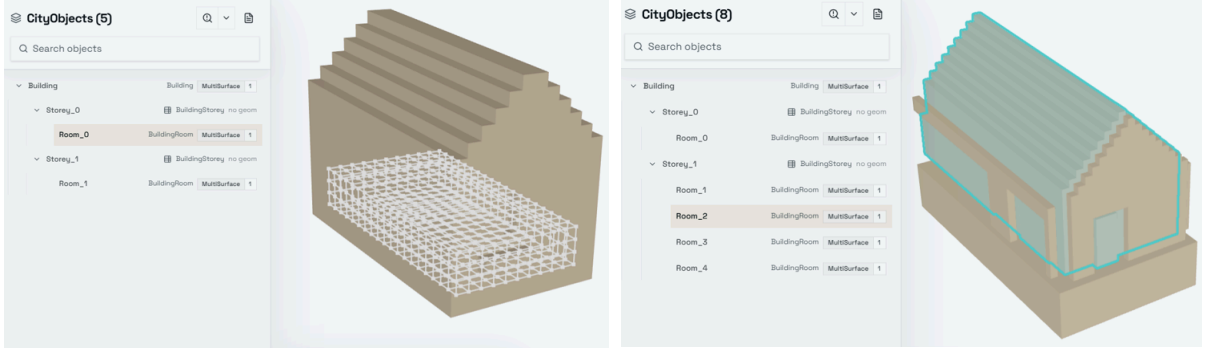


Figure 3: Result of the voxel-based surface extraction (Section 4.4.2) for the House. Left: at voxel size 0.5m, the output again contains 5 CityObjects, matching the two real rooms. Right: at 0.25m, three additional rooms appear; there are now rooms that are not real rooms but the gap inside the exterior wall between its inner and outer layer.

3.2.2. Wellness Center

In Figure 4 we show the results after conversion. Only the relevant elements are kept.

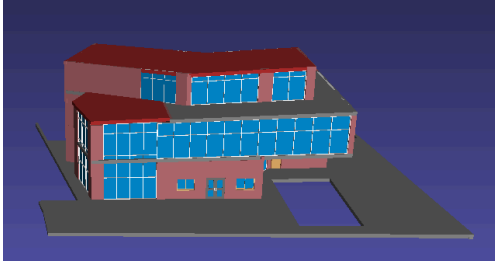


Figure 4: The Wellness Center model after conversion to OBJ. Only elements relevant to the building envelope are kept; furniture, site context, and street-level elements are excluded (Section 3.1).

Figure 5 shows the results of the mesh-based room extraction at voxel size 0.5m. On the left the exterior shell, on the right the detected rooms. Figure 9 in the discussion is a close-up of the small rooms that are deformed. Their shapes are distorted and do not directly match any real room in the building.

Figure 6 shows the result of the voxel-based surface extraction with flood fill, also at voxel size 0.25. Note that in the left image, many of the rooms found here are not real rooms. They are the gaps inside the building's walls, between the inner and outer

wall layers, the same effect we saw for the House in Figure 3. Therefore the filtering approach as described in Section 2.5 is applied after the flood fill and rooms with a very low voxel height are converted to solid (i.e. the shell).

This final approach leaves us with 54 rooms. The height filter removes about a third of the false rooms found at 0.25m. This is still above the building's true room count — which we manually counted to be 49 — since the height filter only catches gaps that are flat in Z — wall-cavity gaps spanning the full room height would not be removed by it. We did not separately verify which of the remaining 54 are genuine rooms versus such cavities.

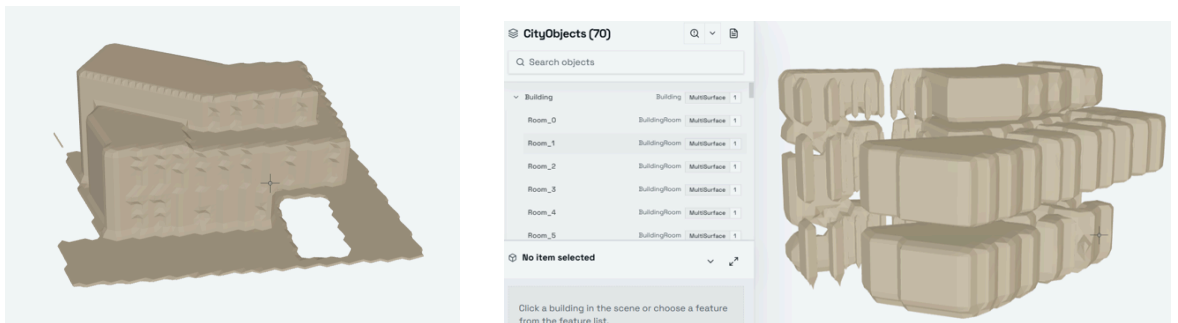


Figure 5: Result of the mesh-based room extraction (Section 4.4.3) for the Wellness Center at voxel size 0.5m, producing 69 rooms.

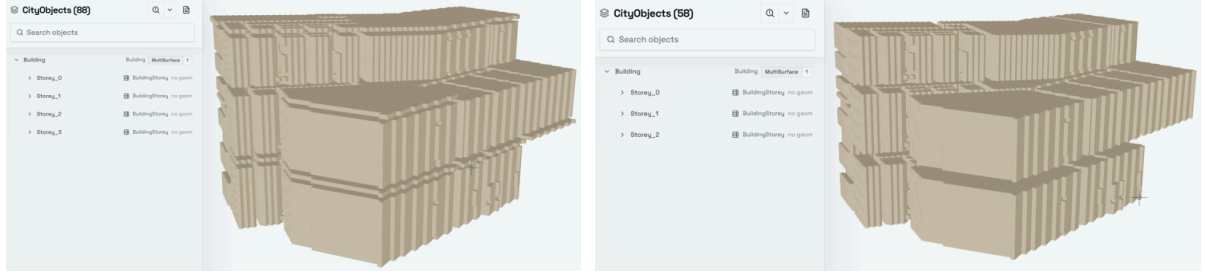


Figure 6: Result of the voxel-based surface extraction (Section 4.4.2) for the Wellness Center at voxel size 0.25m, grouped into storeys. Left: unfiltered result with 83 detected rooms. Right: after the height filter from Section 2.5, 29 flat pseudo-rooms are removed, leaving 54 rooms (Table 1).

Method	Voxel size	Rooms detected
Mesh-based extraction	0.5 m	69
Mesh-based extraction	0.25 m	83
Voxel-based extraction, unfiltered	0.25 m	83
Voxel-based extraction, after height filter	0.25 m	54 (29 removed)

Table 1: Number of detected rooms for the Wellness Center under different extraction methods, all at voxel size 0.25m.

4. Discussion

This section discusses the main design choices in our pipeline and the trade-offs we found along the way. We focus on voxelisation, isosurface extraction, and room detection, since these are the steps where different approaches gave clearly different results.

4.1. Voxel Data Structure

We use the VoxelGrid structure given in the assignment. It stores all voxels in a single 1D array. Each voxel holds one unsigned int. We use this value for two different purposes at different stages of the pipeline: first as a binary solid/empty flag during voxelisation, and later as a region ID (exterior, building, room) after the flood fill.

4.2. Voxelisation

There are 2 possible intersection targets depending on the desired connectivity we want to achieve. We find that using full voxel as a target helps achieve 6-connectivity and is better suited for meshing, but significantly reduces the size of the rooms. Using the 3 axes of a voxel for 26-connectivity is the better approach if we are not trying to convert geometry to mesh later.

Another question is the size of a voxel: a smaller voxel better captures rooms but can also result in “fake rooms” being generated. We find that for the Wellness Centre, a voxel size of 0.25m works best but can produce unwanted rooms that need to be filtered out.

4.3. Isosurface Extraction

We have attempted to extract the isosurface with 2 methods: dual contouring and marching cubes, in both cases relying on CGAL’s implementation of the methods. We find that dual contouring is sensitive to the definition of the gradient function and can produce artefacts even for relatively simple geometry.

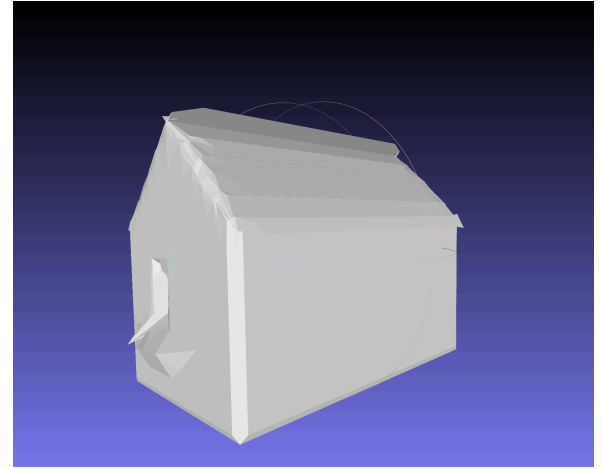


Figure 7: OpenHouse with Dual Contouring

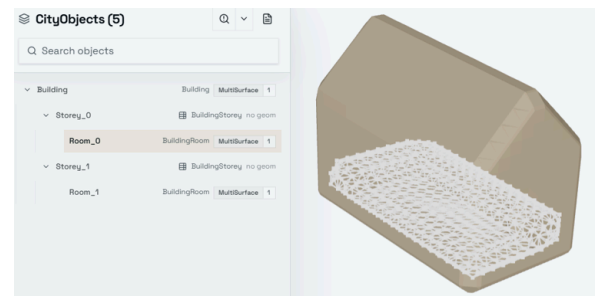


Figure 8: OpenHouse with Marching Cubes

Marching Cubes generally solve this problem but still have others:

1. They are sensitive to the input – if a building is voxelised with 26-connectivity, ambiguous cases are created which are impossible to resolve;
2. Using 6-connectivity significantly reduces the size of non-axis-aligned rooms and may split them into several, as seen in Figure 9.

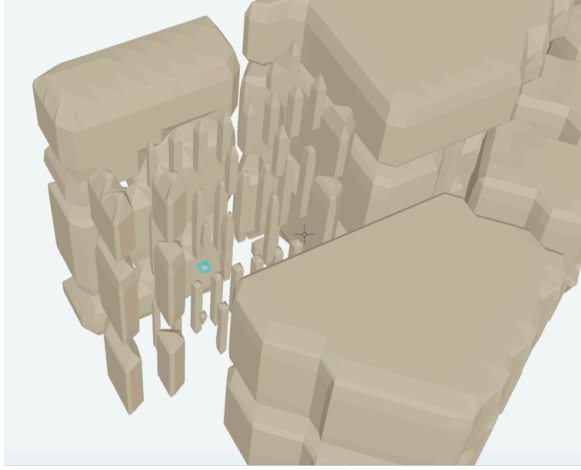


Figure 9: Deformed rooms with marching cubes

4.4. Detecting Rooms

We have attempted 3 different approaches for detecting interior spaces: a ray shooter; flood filling; and extracting them from the mesh.

4.4.1. Shooting Rays

Our initial approach was to detect interior spaces by shooting rays from the bottom voxel and counting how many boundaries they cross:

SHOOTING RAYS

Input: grid: 3D-array of IDs corresponding to voxelised objects, where 0 is unvisited,
 1 is part of building
Output: 3D-array of IDs corresponding to voxelised objects
 2 region ID: $i \leftarrow 2$
 3 $x_{\max}, y_{\max}, z_{\max} \leftarrow$ dimensions of voxel grid
 4 **for** $x \in [0, x_{\max}] \mid \mathbb{N}$ **do**:
 5 **for** $y \in [0, y_{\max}] \mid \mathbb{N}$ **do**:
 6 room ID: $i \leftarrow 0$
 7 **for** $z \in [0, z_{\max}] \mid \mathbb{N}$ **do**:
 8 $v \leftarrow$ query grid[x, y, z]
 9 **if** grid[x, y, z] = 0 **do**:
 10 | grid[x, y, z] $\leftarrow i$
 11 **else do**:
 12 | $i \leftarrow i + 1$
 13 **for** $z \in [0, z_{\max}] \mid \mathbb{N}$ **do**:

SHOOTING RAYS

```

14 | | |  $v \leftarrow$  query grid[ $x, y, z$ ]
15 | | | if grid[ $x, y, z$ ] =  $i$  do:
16 | | | | grid[ $x, y, z$ ]  $\leftarrow 0$ 

```

The general assumption is that the building rooms are horizontally connected; the floors are perfectly stacked; and that the 1-voxel-wide padding ensures that the exterior is fully connected around the building's geometry.

However, there are limitations to this approach. It does not ensure the horizontal connectivity of the rooms, and only requires that a room is bounded vertically.

4.4.2. Flood-filling

To circumvent these limitations, we have decided to move to flood filling the voxel grid instead. This allows us to detect continuous regions of voxels, separated by voxel walls. It is currently our preferred approach as it universally applicable regardless of the geometric complexity.

4.4.3. Extracting Rooms from Mesh

The final approach we tried was to first mesh the voxelisation of the building and extract rooms from it. This approach felt most appropriate for the particular input and output used for this assignment: a single building (i.e. a single outer shell) with one or more interior spaces. The steps are to:

1. construct an isosurface (of class SurfaceMesh) using the implementation of marching cubes in CGAL;
2. extract components of building mesh;
3. assign the largest (as determined by the size of its bounding box) component as exterior, and the smaller ones as interiors.

This implementation relies on the fact that isosurfaces extracted with CGAL's marching cubes contain several connected components of a single mesh.

This approach outputs:

1. An exterior surface
2. A list of interior surfaces.

For a more complex input, such as 2 buildings, this approach would not scale, as the exterior of a smaller building will be treated as a room of a larger one. A possible modification is to perform the flood fill of the exterior and extract 2 isosurfaces: 1 at value 0.5 (representing the interior) and one at value 1.5 (representing the exterior). We can then separate individual

buildings — but this is beyond what the assignment requires.

4.4.4. Summary

Comparing the two final candidate approaches, both the mesh-based and the flood-fill/voxel-based extraction give reasonable results once the voxel size is small enough. At 0.25m, both approaches give similar results before filtering (Table 1). However, only the flood-fill/voxel-based approach lets us filter this result afterwards effectively. Because each room is a labelled set of voxels with a measurable bounding box, we can identify and remove flat pseudo-rooms and bring the count down to 54 rooms. The mesh-based approach has no equivalent step. Its “rooms” are mesh components, not labelled regions, so there is nothing to measure or filter by. Beyond filtering, the flood-fill/voxel-based approach also avoids the “largest component = exterior” assumption, which assumes the building’s solid voxels form a single connected mass and breaks down for inputs with multiple disconnected solid parts or multiple buildings (Section 4.4.3). For these reasons, we consider the flood-fill/voxel-based approach the more robust of the two, even though both produce broadly similar room counts before filtering. This is also why our final CityJSON output is generated using the flood-fill/voxel-based pipeline. Additionally, we provide the CityJSON output for the mesh extraction approach for voxel-size 0.25m and 0.5m.

4.5. Analyzing Performance

In Table 2 we compare the runtime of each pipeline stage for the Wellness Center at voxel sizes 0.25m and 0.5m, using the flood-fill/voxel-based extraction approach.

Stage	0.25 m	0.5 m
Voxelisation	5359 ms	1967 ms
Flood fill + filtering	281 ms	39 ms
Surface extraction	1849 ms	407 ms
CityJSON writing	1732 ms	453 ms
Total	15590 ms	4268 ms

Table 2: Runtime per pipeline stage for the Wellness Center, by voxel size.

Halving the voxel size roughly doubles the grid resolution in each dimension, so an 8-fold increase in the voxel count. The runtime increases by a comparable factor. Voxelisation is the most expensive stage, more than a third of the total. Surface extraction and CityJSON writing both iterate once per voxel face and once per output vertex respectively and scale with the

number of voxels. Flood fill and pseudo-room filtering remain cheap in absolute terms at both resolutions.

4.6. Further Work

We believe further work may be done to find a robust method for converting a voxelised object to a surface. Current approach is only applicable when the input file has one building (i.e. one outer shell) and is not scalable.

Another issue is that the files are relatively large as a planar surface is represented as multiple quads. We suggest that a geometry may be simplified if the intermediate vertices are removed and only the full outer/inner rings are stored.

5. Work Distribution and AI

5.1. Work Distribution

RV implemented OBJ reader and voxelisation, as well as trialled the ray-shooting approach for room detection. AK implemented the initial code for flood filling and surface extraction. This code was subsequently adapted by MC. MC implemented the final version of these functions alongside CityJSON writing.

Each group member wrote the sections of the report corresponding to their work. Additionally, AK wrote the Introduction and Discussion sections and supplied pseudocode; and MC wrote Evaluation section.

5.2. AI Statement

- AK hates to admit it but the majority of the code he supplied was generated — vibecoded? — by Claude 4.6 Low, used in a browser GUI. However, all the writing is fully his.
- MC used Claude to support in writing parts of the code and C++ syntax. Parts of the writing were reformulated with the use of AI.
- RV only used AI assistance to interpret specific semantic properties and the syntax of the OBJ file format. The remainder of the code and the entirety of the text were written manually.